

Trabalho Final (A1) - Sistema de Controle de Estacionamento

Vale 50% da nota - Entrega 14/06 - Defesa 16/06/2026

Prof. Dr. José Jairo de Santana e Silva

Programação de Computadores - Universidade Positivo - 2026/1

Informações gerais

- **Avaliação:** A1 (vale 50% da nota final).
- **Entrega do .zip:** 14/06/2026 (domingo), até 23:59.
- **Defesa:** 16/06/2026 (Aula 15).
- **Formato de trabalho:** em trios.
- **Linguagem:** Python 3 puro. Não usar pandas, frameworks ou interface gráfica.
- **Entrega:** código-fonte (.py), README e três casos de teste documentados.

Objetivo

Construir um **Sistema de Controle de Estacionamento** em Python, com menu interativo, que permita registrar a entrada e a saída de veículos, listar os veículos estacionados, consultar vagas disponíveis e calcular o valor a pagar na saída.

O programa deve rodar **apenas em memória**. Não precisa salvar dados em arquivo. Quando o programa fechar, os dados podem ser perdidos.

Estrutura de dados sugerida

Cada veículo deve ser representado por um **dicionário**. O conjunto de veículos estacionados deve ser uma **lista de dicionários**:

```
veiculos = [  
    {  
        "placa": "ABC-1234",  
        "entrada": "08:30",  
        "vaga": 1,  
        "tipo": "carro"  
    }  
]
```

Funcionalidades obrigatórias

O programa deve apresentar um menu com pelo menos estas opções:

1. **Entrada de veículo:** registra um veículo novo no estacionamento.
2. **Saída de veículo:** remove um veículo e calcula o valor a pagar.
3. **Listar veículos estacionados:** mostra a tabela atual.
4. **Consultar vagas disponíveis:** quantas vagas estão livres.

5. **Encerrar:** sai do programa.

O menu deve ficar **em laço** (`while`) até o usuário escolher encerrar.

Regras do sistema

- O estacionamento tem **capacidade limitada** (defina como constante no código, por exemplo `CAPACIDADE_MAXIMA = 10`).
- Não pode cadastrar veículo se o estacionamento estiver **lotado**.
- Não pode cadastrar a **mesma placa duas vezes** ao mesmo tempo.
- Na saída, o programa deve verificar se a placa **está cadastrada**.
- O sistema deve calcular o **tempo de permanência** em minutos.
- O sistema deve calcular o **valor a pagar** segundo a tabela abaixo.

Tabela de cobrança

- **Primeira hora** (até 60 min): R\$ 5,00.
- **Cada 15 minutos adicionais**, ou fração: R\$ 2,00.

Exemplos:

Permanência	Valor
30 min	R\$ 5,00
60 min	R\$ 5,00
75 min	R\$ 7,00
90 min	R\$ 9,00
120 min	R\$ 13,00

Atenção: 76 minutos já entram no bloco seguinte (R\$ 9,00), porque conta cada bloco **iniciado** de 15 min.

Validações de entrada (parte essencial do trabalho)

Esta é a parte mais cobrada na avaliação. O programa **nunca** deve quebrar por causa de entrada errada. Em todo `input()` em que o usuário pode errar, faça uma **validação** e mostre uma **mensagem clara** explicando o que aconteceu. **Nunca** mostre traceback do Python para o usuário final.

Validações mínimas exigidas

Situação	Mensagem que deve aparecer
Placa fora do formato AAA-1234	Placa invalida. Use o formato ABC-1234.
Placa já cadastrada (na entrada)	Essa placa ja esta cadastrada no estacionamento.
Placa não encontrada (na saída)	Placa nao encontrada no estacionamento.
Horário fora do formato HH:MM	Horario invalido. Use o formato HH:MM (exemplo 08:30).
Hora maior que 23 ou minuto maior que 59	Horario invalido. Hora entre 00 e 23, minuto entre 00 e 59.
Horário de saída anterior ao de entrada	O horario de saida nao pode ser anterior ao de entrada.

Situação	Mensagem que deve aparecer
Estacionamento lotado (na entrada)	Estacionamento lotado. Nao foi possivel cadastrar.
Opção do menu inválida	Opcao invalida. Escolha um numero do menu.
Tipo de veículo fora de “carro” / “moto”	Aceitar mas registrar como carro (padrão)

Como o programa deve se comportar quando há erro

Sempre que uma validação **falhar**, o programa precisa:

1. **Mostrar a mensagem** correspondente.
2. **Não** alterar a lista de veículos.
3. **Voltar ao menu** para o usuário tentar de novo.

Não é permitido:

- Deixar o programa quebrar com `ValueError`, `KeyError`, `TypeError`, etc.
- Cadastrar dados inválidos na lista e “consertar depois”.
- Mostrar mensagem de erro técnica (com palavras como `Traceback`, `Exception`, nomes de funções).

Como implementar

Para cada validação, escreva uma **função separada** que devolve `True` ou `False`:

```
def validar_placa(placa):
    """Devolve True se a placa esta no formato AAA-1234."""
    placa = placa.upper().strip()
    if len(placa) != 8:
        return False
    if placa[3] != "-":
        return False
    letras = placa[:3]
    numeros = placa[4:]
    if not letras.isalpha():
        return False
    if not numeros.isdigit():
        return False
    return True
```

E use no fluxo principal:

```
placa = input("Placa (formato ABC-1234): ").strip().upper()
if not validar_placa(placa):
    print("Placa invalida. Use o formato ABC-1234.")
    return # ou continue, dependendo do contexto
```

Toda função de validação deve ter **docstring** explicando o que ela verifica.

Funções obrigatórias

O programa **não pode** ser um bloco gigante. Divida em funções pequenas, cada uma com **uma responsabilidade**. No mínimo:

```
def validar_placa(placa):          pass
def validar_horario(horario):      pass
def calcular_valor(minutos):       pass
def cadastrar_veiculo(veiculos):   pass
def remover_veiculo(veiculos):     pass
def listar_veiculos(veiculos):     pass
def consultar_vagas(veiculos):     pass
def main():                        pass
```

A função main() controla o menu. O fim do arquivo deve ter:

```
if __name__ == "__main__":
    main()
```

Exemplo de execução (telas esperadas)

Menu principal

```
=====
ESTACIONAMENTO - MENU
=====
1 - Entrada de veiculo
2 - Saida de veiculo
3 - Listar veiculos estacionados
4 - Consultar vagas
0 - Encerrar
Escolha uma opcao:
```

Entrada bem-sucedida

```
Escolha uma opcao: 1
Placa (formato ABC-1234): ABC-1234
Horario de entrada (HH:MM): 08:30
Tipo do veiculo (carro / moto): carro
Veiculo ABC-1234 cadastrado na vaga 1.
```

Entrada com placa inválida

```
Escolha uma opcao: 1
Placa (formato ABC-1234): abc1234
Placa invalida. Use o formato ABC-1234.
```

Saída com cálculo de valor

```
Escolha uma opcao: 2
Placa do veiculo que esta saindo: ABC-1234
Horario de saida (HH:MM): 09:45
```

```
Placa:          ABC-1234
Entrada:        08:30
Saida:          09:45
Permanencia:    75 min
Total a pagar:  R$ 7,00
Vaga 1 liberada.
```

Tentativa de cadastrar placa repetida

```
Escolha uma opcao: 1
Placa (formato ABC-1234): ABC-1234
Essa placa ja esta cadastrada no estacionamento.
```

O que NÃO precisa fazer

Para deixar o escopo claro, **não** é necessário:

- Salvar dados em arquivo (.txt, .csv, .json).
- Usar `pandas`, `numpy` ou bibliotecas externas.
- Criar interface gráfica.
- Usar classes e Programação Orientada a Objetos.
- Implementar login, usuários ou permissões.
- Suportar placa Mercosul (formato ABC1D23). Use apenas AAA-1234.

Entrega

Cada trio deve entregar:

1. **estacionamento.py** com o código completo.
2. **README.md** descrevendo:
 - Nome dos três integrantes do trio.
 - Como rodar o programa.
 - Quais funções foram implementadas.
 - Decisões de projeto (capacidade adotada, tipos aceitos, etc.).
3. **casos_de_teste.md** com pelo menos **três casos** descritos no formato:

```
Caso 1 - Cadastro bem-sucedido
Entrada: opcao=1, placa=ABC-1234, horario=08:30, tipo=carro
Saida esperada: "Veiculo ABC-1234 cadastrado na vaga 1."
```

```
Caso 2 - Placa invalida
Entrada: opcao=1, placa=ABC1234
Saida esperada: "Placa invalida. Use o formato ABC-1234."
                A lista de veiculos permanece igual.
```

```
Caso 3 - Calculo de valor para 75 minutos
Entrada: entrada=08:30, saida=09:45 (apos cadastro previo)
Saida esperada: Total a pagar: R$ 7,00
```

A entrega deve ser feita **até 14/06/2026 (domingo), 23:59**, em um único **.zip** com nome **A1_Sobrenome1_Sobrenome2_Sobrenome3.zip**. Entregas após esse prazo não serão aceitas.

Defesa

No dia da defesa (16/06), o trio terá cerca de **10 minutos** para:

1. Mostrar o programa rodando (menu, cadastro, saída com cobrança).
2. Demonstrar **pelo menos duas validações** funcionando (placa inválida e estacionamento lotado, por exemplo).
3. Explicar **uma função** escolhida pelo professor.

O professor poderá pedir para o trio **modificar algo no código ao vivo** (por exemplo: mudar a capacidade, alterar a tabela de preços). O trio deve conseguir. Todos os integrantes precisam estar presentes e participar.

Critérios de avaliação

Critério	Peso
Funcionamento correto do menu e das 5 operações	30%
Validações de entrada e mensagens de erro claras	20%
Modularização (funções com uma responsabilidade) e nomes claros	15%
Uso adequado dos conteúdos da disciplina (listas, dicionários, condicionais, laços, funções)	15%
README e arquivo de casos de teste	10%
Defesa: explicar o código e responder modificações ao vivo	10%

Cronograma sugerido

Data	Etapas
02/06 (Aula 13)	Apresentação do trabalho. Definir trios.
02/06 a 09/06	Esqueleto: menu + funções vazias com pass .
09/06 (Aula 14)	Prova A2.
09/06 a 13/06	Implementação completa, validações, casos de teste, README.
14/06 (domingo) até 23:59	Entrega do .zip.
16/06 (Aula 15)	Defesa.

Dicas finais

- Comece pelo **esqueleto** das funções (todas com **pass**) e pelo menu. Garanta que o menu roda em loop antes de implementar qualquer função.
- Implemente uma operação por vez. Teste cada uma antes de passar para a próxima.

- Escreva e teste as **funções de validação** logo no começo. Elas serão usadas várias vezes.
- Use a função `testes()` (com `assert`) que vocês praticaram na disciplina para validar as funções puras (`validar_placa`, `validar_horario`, `calcular_valor`).
- **Não** invente requisitos não pedidos. O escopo está fechado neste documento.
- Em caso de dúvida sobre o enunciado, perguntem **antes** da defesa.

Checklist final antes de entregar

- ☐ O programa roda sem erros desde o início ao fim?
- ☐ O menu volta sozinho após cada operação?
- ☐ Todas as validações da tabela mostram a mensagem certa?
- ☐ O programa nunca mostra traceback do Python?
- ☐ Há pelo menos uma função para cada operação do menu?
- ☐ As funções têm docstring?
- ☐ Os nomes de variáveis e funções estão claros?
- ☐ O cálculo de valor está conforme a tabela?
- ☐ O README está completo?
- ☐ Os três casos de teste estão descritos?
- ☐ O arquivo `.zip` tem o nome no formato pedido?